

Block chain

Cosa è una moneta virtuale? È un qualcosa che può essere utilizzata per effettuare delle transazioni virtuali.

Le monete reali hanno un qualche valore intrinseco (oro) o stabilito (la banconota ha un prezzo stabilito dalla banca). Generare una copia di un qualcosa che ha una valuta intrinseca ha un costo quanto il valore intrinseco stesso. Invece, per quanto riguarda la banconota la copia deve essere in qualche modo complicato da fare (cioè devi far sì in maniera artificiale che il costo della copia abbia il valore stesso della moneta)

Fare copie di un qualcosa digitale è molto semplice e, dunque, è molto difficile impedire la copia di una sorta di valuta virtuale.

Pagare in digitale con carte di credito ecc. oltre che a “inviare dati digitali”, in genere i TTP si occupano di occuparsi delle “scartoffie” reali legate alla valuta reale.

Bitcoin -> Satoshi Nakamoto.

Bitcoin non ha TTP, è un sistema distribuito, c'è pseudoanonimato e, all'inizio, costava poco fare transazioni.

03/03/2026

Funzione hash $H: D \rightarrow R$ con $|D| > |R|$.

Collisione $H(x_1) = H(x_2)$ ecc. (VEDI CRITTOGRAFIA)

In bitcoin D ha una taglia circa “illimitata” e R è a 256 bit. SHA2.

Puzzle friendly, cioè è possibile creare una “sfida” computazionalmente interessante.

Hiding:

1. Dato $H(x)$ non voglio sia possibile ottenere x
2. Voglio che anche se si conosca x iniziale e un hash, non voglio sia possibile capire se x corrisponde a quell'hash. In una puntata posso capire se x non è uno specifico valore.

Commitment e hiding fino a quando non vengono verificate. Binding, una volta pubblicato l'hash non è più possibile cambiare idea (perché stiamo usando hash).

05/03/2026

Utilizzeremo solo funzioni hash collision resistant

Commitment is a thing that you just put somewhere and what's inside the commitment post, is blocked and u can't change it.

We have two algorithms that are **Commit and Verify**. Commit stands for the algorithm that needs in input the message and the randomness and makes them bind. The Verify is an algorithm that indicates us if the message and the randomness are correct outputting 1 if the message and the randomness are correct respect of the commitment algorithm, 0 otherwise.

Puzzle-friendly is a property of an hash function that makes it “playable” in sense of finding a certain output. Thanks to the hash function property that is something like $H(k|x) = y$, is impossible to find in reverse an $k|x$ that $H(k|x)$ is y .

Is very difficult to find it but it's very easy to verify.

In general, it requires a lot of time to solve the puzzle but when it's complete it's easy to verify that is correct. In a blockchain system, we usually use the hash function in a way that we want a certain y in output that has a certain pattern that is the puzzle that we want to solve. In our case we want that y in output has a certain bit set on 0 like the first 60 bit are 0.

Many zeros you want, much difficult the puzzle will be.

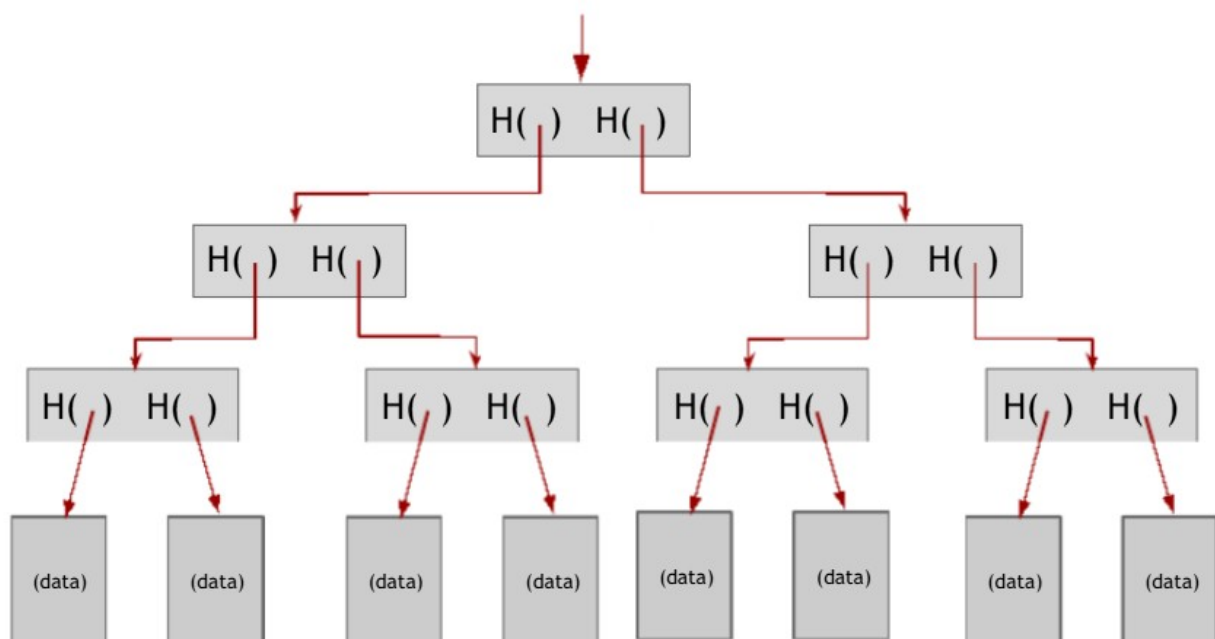
A blockchain is a sort of linked list.

We have a hash pointer. We have a block that points to a data, but we have also the hash of the data of the previous block that's guaranteed integrity of the previous block because if someone modify the data of the previous block, the hash will change (because we assume that the hash function is collision resistant)

Even if you want to change one block of the chain, all the hash are different.

In a blockchain you have a ledger that is distributed on all the peer of the blockchain system. But instead of conserve all the records you have just the last one that can prove the integrity of all the previous record because one block guarantees that the previous block is integer.

The hash pointers in the blocks are called Merkle Trees. This kind of trees have the data on leaf, and the next level are the hash of the concatenation of the previous one. This trees permits us to check in a logarithmically way the integrity of all data. We construct the trees in a bottom up way so that we will have always balanced tree.



Data on leaf are ordered in a certain way to make the tree usable with a sort of dictionary research. We can use hash pointers to guaranteed integrity of a data structure till that structures have no cycle.

Digital Signatures

A digital signature is something that only you can put but anyone can verify. A signature is tied to a particular document and guaranteed the integrity of the document. We have a sign key to put the sign on a document and we have a verify key

that is used to verify the validity of the sign for the document. We have an algorithm for sign and an algorithm to verify. We require that the sign and verify algorithm are correct and that is impossible to forge a signature if you don't have the sk.

Your private key, your bitcoin.

You can create a lot of keys on your own. In a transaction you just put your "keys" but not your identity so a lot of people thinks that you are anonymous. But we will see that's not true.

Bitcoin guarantee just pseudonymity (you can change your name as many times as you want because you can create a lot of public key).

10/03/2026

ECDSA

È uno schema di firma particolare che sfrutta punti di curve ellittiche che permettono chiavi crittografiche molto piccole. In questa trattazione parleremo di DSA per semplicità ma ECDSA è la stessa cosa solo con delle accuratezze matematiche in più.

DSA sfrutta il problema del logaritmo discreto. Vedi crittografia.

Le coppie di chiavi non sono certificate da qualche parte per garantire l'anonimato. Ognuno è garante della propria chiave in un certo senso (a nessuno conviene dare chiavi "fallate").

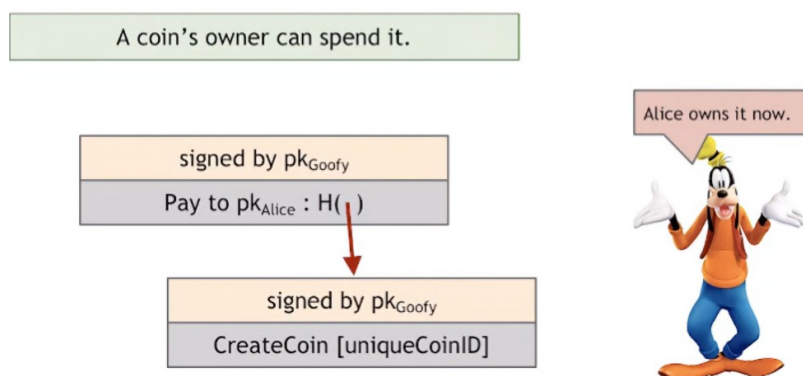
Come già detto tutto fa riferimento alle chiavi pubbliche che saranno le identità chiamate **indirizzi** in bitcoin.

Cryptocurrencies semplici

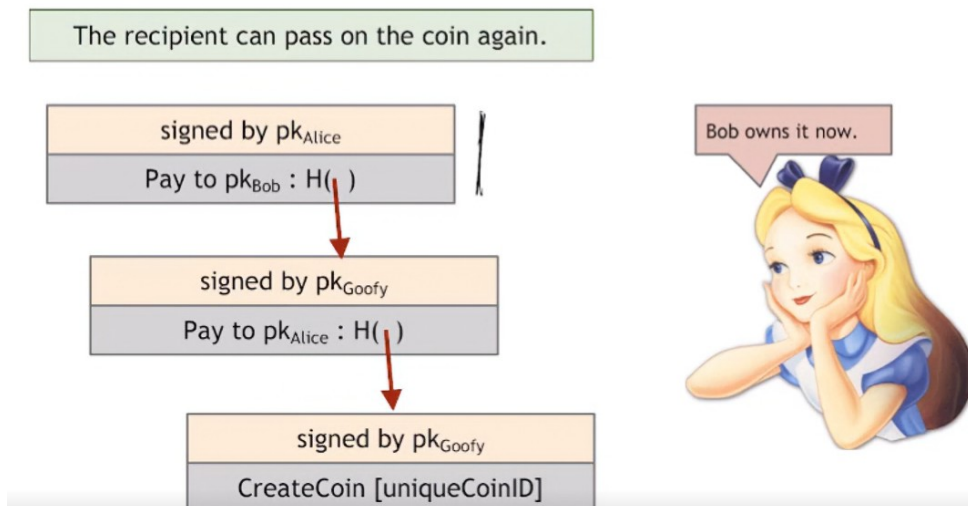
GoofyCoin

Pippo crea una nuova moneta con un ID univoco tramite CreateCoin e dice "okay questa moneta appartiene a me" e **firma** questa transazione per far sì che tutti sappiano che quella moneta gli appartiene.

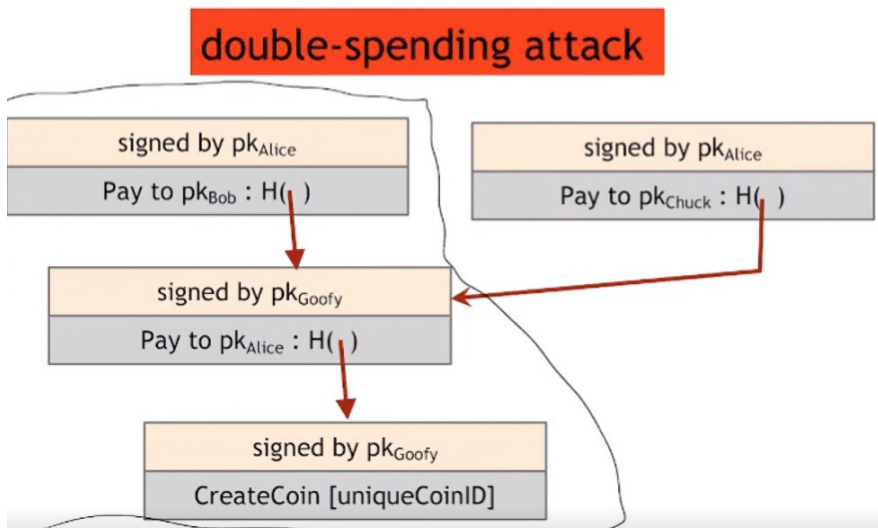
Supponiamo esista l'operazione **pay** che dà dei coin a un certo indirizzo. Crea l'hash pointer della moneta creata precedentemente così da trovare e verificare la moneta per poi fare il pagamento.



A sua volta alice potrà spendere le monete appena ricevute mettendo l'hash pointer della transazione appena avvenuta con Pippo.



Problema è che a livello di efficienza più transazione abbiamo più questa catena di controlli è lunga. Inoltre, esiste il problema del **double spending** per cui alice effettua due pagamenti con la stessa “moneta”.

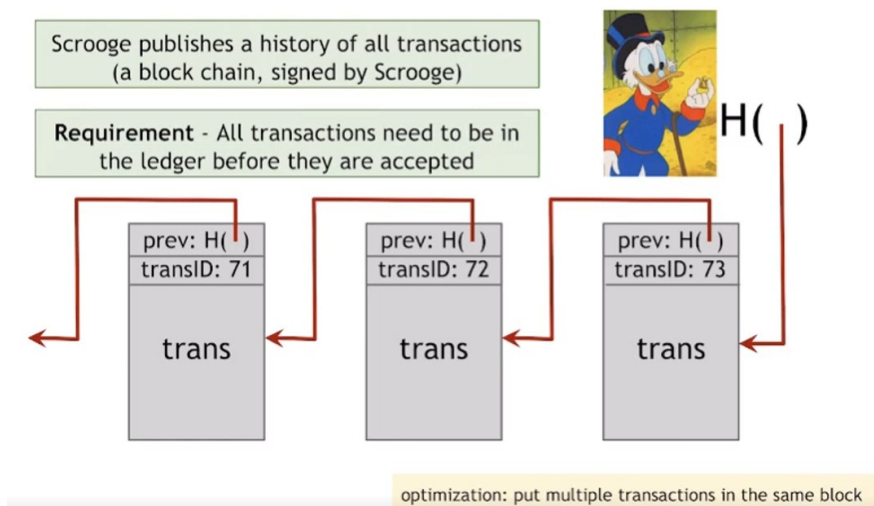


Questo è il problema più grande delle criptovalute oltre al fatto che ognuno può creare le monete aggratis.

Il problema nasce dal fatto che noi vogliamo un sistema del tutto decentralizzato.

ScroogeCoin

A differenza del precedente sistema, abbiamo che Zio Paperone è colui che gestisce tutta la blockchain e conferma tutte le transazioni.



Immaginiamo che l'unico che può creare nuove monete è Zio Paperone e una transazione è valida se Zio Paperone dice che lo è.

Anziché fare pagamenti unitari, possiamo spezzettare i pagamenti facendo sì che, ad esempio, se l'unità è 100 euro, per pagare un caffè pago 1 euro al commerciante e 99 euro a me stesso. Naturalmente ciò complica le cose perché si devono fare tante verifiche, tuttavia è fattibile. Elimina il double spending perché Zio Paperone non se fa fregare e verifica se ci sono due transazioni in contraddizione. Il problema è se Zio Paperone non è onesto. Poiché ha tutto il potere, nessuno può contrastarlo.

Come risolviamo il problema di centralizzazione vs decentralizzazione

Questo è un problema molto generico che si trova ovunque e in realtà non è detto che tutto ciò che è decentralizzato non è anche in qualche modo centralizzato. Es. gmail ecc.

Per quanto riguarda Bitcoin abbiamo:

1. Chi gestisce la blockchain (il ledger)?
2. Chi ha l'autorità per dire quali operazioni sono valide?
3. Chi crea nuove monete?
4. Chi determina le nuove regole e come?
5. Che valore ha Bitcoin rispetto al vero valore di mercato? Come si scambia con altre monete ecc.

Altre questioni riguardano i wallet, il come effettuare le transazioni ecc.

Alcuni aspetti decentralizzati di Bitcoin sono:

1. Il network Peer to peer
2. Mining
3. Updates del software

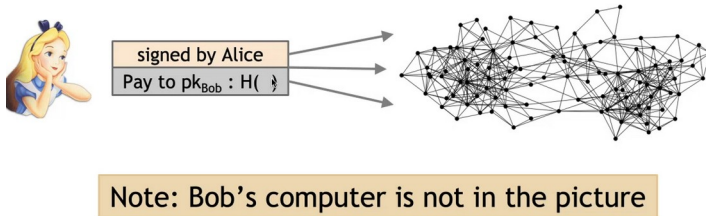
Tutte queste operazioni sono fattibili da chiunque (anche se in realtà spesso non è vero...)

Distributed Consensus

Vogliamo che la maggior parte dei nodi appartenenti alla rete possano eseguire un protocollo per determinare lo stesso risultato come, ad esempio, il prossimo nodo da aggiungere alla catena.

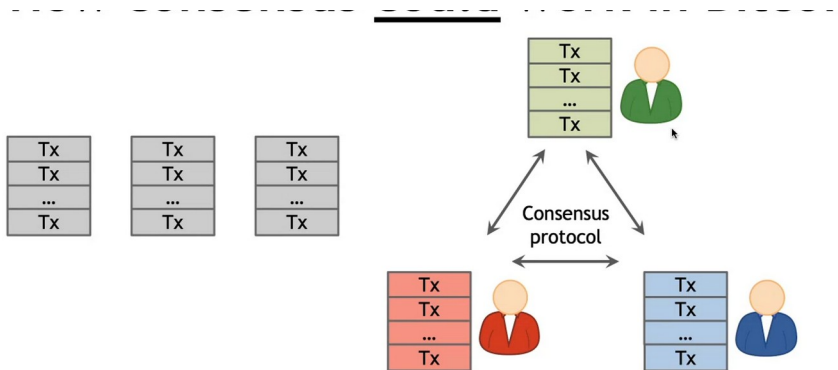
Ad esempio, per fare un pagamento alice manda broadcast a tutti i nodi la sua transazione cosiché la verifichino.

When Alice wants to pay Bob:
she broadcasts the transaction to all Bitcoin nodes



Possiamo notare che Bob non fa parte della rete, non è necessario che lo sia. Tantomeno lo è Alice.

In ogni momento, ogni nodo ha dei blocchi che considera validi (non necessariamente tutti i nodi hanno gli stessi blocchi in questa fase) e tutti i nodi della rete hanno però un set di operazioni sottostanti che considerano valide.



Supponiamo che rosso blu e verde sono tre nodi che propongono il loro set di operazioni che devono essere valide. Per accertarci della cosa, tutti verificano i blocchi proposti e verificano che siano validi. Prima di questo, si sceglie chi deve pubblicare e poi si verifica.

Il problema è che i nodi potrebbero crashare o potrebbero esserci dei nodi malevoli. Inoltre, possiamo avere problemi di latenza, non tutti i nodi sono connessi e potrebbero anche esserci problemi di rete.

Per risolvere questo problema, prendiamo ispirazione dal problema dei generali bizantini che dovevano scambiarsi messaggi di vitale importanza. Il consenso di nodi deterministici non può esserci se anche un solo nodo è malevolo. (così si diceva)

Teniamo in considerazione un modello in cui tutti i nodi siano razionali, ossia che prendano delle scelte intelligenti poiché magari conviene seguire il protocollo grazie a degli incentivi perché magari guadagnano bitcoin.

Inoltre, a noi non interessa il consenso immediato, ci interessa il consenso ogni circa un'ora. E ancora, sfruttiamo la randomness.

Un approccio utile sarebbe quello di poter risalire a un'identità reale per questioni di sicurezza così possiamo farci un'idea su quella specifica identità (possiamo identificare nodi malevoli). Tuttavia, questo approccio non è fattibile in bitcoin perché le chiavi

pubbliche sono le identità e ognuno può crearne quante ne vuole poiché l'obiettivo di Bitcoin è quello di garantire **pseudoanonimato**.

Sybil Attack un attaccante crea tanti nodi diversi malevoli tutti sotto il suo controllo.

Il nostro punto di partenza sarà che il prossimo nodo a scegliere il blocco da inserire nella catena sarà fatto in maniera randomica tramite una sorta di lotteria. Ad ogni round, scegliamo un nodo a caso che vince e diventa "leader" cioè sceglie il blocco da aggiungere. Gli altri nodi non possono influire sulle scelte del leader ma possono scegliere se estendere o ignorare il blocco da lui proposto. A lungo termine, la catena si allungherà in maniera valida.

12/03/2026

Assumiamo (per ora) che otteniamo il consenso tramite una lotteria scegliendo il nodo a caso che sceglierà il blocco. Se colui che si sceglie è disonesto, nessuno se lo caga e estendono da dove decidono loro. Questo viene detto consenso implicito. Tutti gli altri vedono la fork e scelgono.

Descriviamo il semplice algoritmo di consenso:

1. Le nuove transazioni vengono postate broadcast.
2. Ogni nodo inserisce le transazioni in un blocco.
3. Chi vince la lotteria pubblica il suo blocco
4. Altri nodi lo accettano solo se il blocco è valido.
5. I nodi esprimono il consenso accettando il blocco inserendo l'hash pointer alla fine della catena che come ultimo blocco quello inserito.

Cosa potrebbe fare un nodo malevolo?

1. Provare a rubare bitcoin? Ogni transazione è firmata con la chiave privata di colui che deve inviare bitcoin.
2. DoS contro un particolare utente? Immaginiamo che un nodo voglia ignorare tutte le transazioni rivolte ad un particolare utente, può farlo? Prima o poi qualcuno la metterà se la rete è abbastanza grande. È un'assunzione da fare a prescindere da questo attacco poiché una moneta ha senso solo se c'è molta partecipazione al sistema (cioè se molti la vogliono).
3. Double spending attack? Alice fa una transazione verso Bob che viene aggiunta come ultimo blocco. Alice vince la lotteria e pubblica il prossimo blocco con una nuova transazione in cui paga Charlie ignorando l'ultimo in cui c'è il pagamento verso Bob. Di fatto c'è un double spending perché potrebbe essere estesa una o l'altra. Si estende sempre la catena più lunga, quindi in pratica noi vedremo che la catena è unica (non c'è double spending), ma in realtà c'è stata la truffa poiché Alice ha ricevuto il bene da Bob. Per evitare questo problema, Bob aspetta finché il blocco con la sua transazione non viene esteso 6 volte.

Quando un nodo stora le sue transazioni, deve assicurarsi che queste non siano double spending (altrimenti nessuno se li caga).

Il nostro obiettivo sarà penalizzare i comportamenti negativi. Abbiamo 2 incentivi in particolare:

1. Block reward: quando viene inserito un nuovo blocco proposto da noi, si guadagnano bitcoin (soldi). Coin create transaction. Si è deciso che ci debba essere un cap a 21 milioni btc.
2. Le transazioni hanno delle "mance". Il creatore del blocco le guadagna.

Adesso il problema diventa, come prendiamo un nodo a caso evitando che ognuno crei 10 mila account per vincere la lotteria? Come risolviamo il Sybil Attacks? Come evitiamo il free for all?

Proof of work

L'idea è che non scegliamo un nodo a caso, scegliamo un nodo in base a quante risorse questo usa per mantenere la blockchain. Queste risorse devono essere non monopolizzabili e non duplicabili. Nel caso della proof of work si tratta di computing power, nel caso della proof of stake è quante risorse possiedi.

Per la proof of work facciamo sì di scegliere il prossimo nodo o in base alla potenza messa a disposizione o facciamo competere i nodi per il diritto per creare il nodo o facciamo sì che sia difficile creare una nuova identità.

Il problema computazionale sarà l'hash puzzle. Si indovina la nonce. I tx non sono più transazioni vere e proprio ma sono teste dei merkle tree. Supponendo di avere una nonce di 80 bit, cosa accade se per le transazioni che ho scelto, non bastano quegli 80 bit? Sarebbe impossibile in quel caso risolvere il tutto. Infatti, i tx sono teste di merkle tree dove si possono anche inserire nonce per modificare l'hash.

È impossibile minarsela da solo...

Più o meno si aspettano 10 minuti per ogni nuovo blocco. Più persone se la minano, più la challenge diventa difficile.

Il sistema funziona se la maggior parte delle persone che se la minano, lo fanno in maniera onesta.

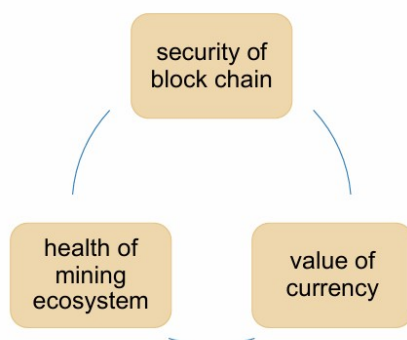
È importante che sia facile verificare che l'hash puzzle sia corretto.

17/03/2026

Vogliamo che un blocco possa essere pubblicato ogni 10 minuti. È necessario che il reward in termini di bitcoin sia una cosa di questo tipo:

If mining reward (block reward + Tx fees)	>	hardware + electricity cost	→ Profit
--	---	--------------------------------	----------

Il sistema ha un bootstrap:



Come facciamo a essere sicuri che il sistema non venga monopolizzato? Cioè, cosa accade se esiste un attaccante che ha una potenza di calcolo superiore al 51% della rete totale? Cosa potrebbe fare?

What can a “51% attacker” do?

Steal coins from existing address? ✗

Suppress some transactions?

- From the block chain ✓
- From the P2P network ✗

Change the block reward? ✗

Destroy confidence in Bitcoin? ✓✓

19/03/2026

MDR

Quello che ha detto catalano. Coin Based transaction non è firmata. L'output potrebbe essere leggermente più piccolo dell'input a causa delle mance ai minatori.

L'indirizzo è l'hash della chiave pubblica (si usa questo perché è più piccolo)

Consolidation compattò gli UTXO in uno solo.

Lock time, la transazione può essere fatta solo se il blocco è lontano almeno 100 dal genesis block.

VEDI REALE TRANSAZIONE

Perché script

Bitcoin script è stack based (non ha memoria né variabili né nulla). Non è turing complete.

Questo script è eseguito dai miner.

Control logic

Poiché manca di memoria, tutti i dati che servono sono nello stack e si devono poppare da lì. È quindi necessario siano inseriti nell'ordine corretto.

OP_DUP duplica il primo elemento dello stack

OP_HASH calcola l'hash del primo elemento dello stack.

OP_RETURN può servire per buttare roba nella blockchain. Timestamping. Checkmultisig.

24/03/2026

Proprietà Nakamoto

Anonimato può essere interpretato in due modi:

1. Senza usare il proprio nome reale.
2. Senza usare alcun nome.

Bitcoin adotta il primo approccio poiché si usano gli hash delle chiavi pubbliche e questo è detto **pseudo anonimato**. Esistono degli attacchi alla pseudoanonimità.

Anonimato è la somma di pseudoanonimato e unlinkability. Con proprietà di Unlinkability indichiamo la proprietà per cui non è possibile andare a collegare tutte le interazioni ad una sola persona.

Perché abbiamo bisogno di unlinkability?

Molti servizi Bitcoin vogliono l'identità reale. (es. wallet) tramite quelli è possibile togliere l'anonimato.

Unlinkability in tre modi:

1. Difficile linkare diverse identità alla stessa persona
2. Difficile collegare diverse transazioni allo stesso user
3. Difficile collegare sender di un pagamento e receiver. (?)

Se il pagamento è una singola transazione, questo è impossibile. Noi guarderemo una sorta di transazione nel complesso. Abbiamo una transazione che viene "spalmata" su più transazioni.

Anonymity Set: set of transaction che l'avversario non può distinguere dalla nostra transazione.

Sarà complicato arrivare all'anonimato.

Perché abbiamo bisogno dell'anonimato in una criptovaluta?

Il problema dell'ottenere questo è che tutto il ledger è pubblico e permanente e, quindi, tutti possono vedere tutto. In banca non possiamo estrarre le transazioni di una persona, per quanto riguarda bitcoin è invece del tutto possibile. Quindi, se non ci fosse anonimato sarebbe possibile, ad esempio, fare spionaggio industriale.

Abbiamo anche degli svantaggi dovuti al fatto che lì si potrebbe fare riciclaggio di denaro o si potrebbe pagare qualcosa di "problematico". Riciclaggio di denaro. Si può controbattere perché ad una certa è necessario tornare alla vera economia.

Possiamo sviluppare qualcosa che prende solo il meglio dell'anonimità ma ne escluda i possibili lati negativi? Questo è chiaramente impossibile. Se uno è anonimo lo è sia quando fa qualcosa di "buono" che quando fa qualcosa di "cattivo". Una tecnologia non è "etica", fa ciò che deve fare.

Un dilemma simile è per TOR che è un sistema che ottiene comunicazione anonima e fa sì che ci sia unlinkability tra sender e receiver. Si usa per cose belle e cose brutte.

Anonimato è facile da ottenere in sistemi centralizzati ma difficile in sistemi distribuiti.

Blind signature

Il receiver sceglie un serial number e lo nasconde. Manda questo messaggio "offuscato" al sender che lo firma. Il receiver deve riuscire ad estrarre la firma da questo messaggio.

Questo si può fare ma abbiamo bisogno di un autorità centrale.

Abbiamo due utenti blu e rosso. Blu vuole prendere dei soldi dalla banca. L'utente sceglie un serial number e lo maschera con un qualcosa e lo fa firmare alla banca. Blu vuole pagare rosso e gli dà il serial number in chiaro con la firma che ha ricevuto. Rosso riporta il serial id alla banca e verifica se questi soldi sono già stati spesi.

La banca non può linkare sender e receiver. Si assume che i pagamenti siano unitari e ci siano tanti utilizzatori.

Il sistema ha questa continua interazione con la banca ed è imprescindibile questa cosa.

Anonimizzazione e decentralizzazione sono molto difficili da ottenere assieme

26/03/2026

Cosa si può fare con gli script.

Real life transaction: A vuole mandare soldi a B senza farsi fregare e B vuole dargli il "bene" senza farsi fregare. Siamo chiaramente in una fase di Stallo.

L'idea è questa:

supponiamo di avere un terzo utente J.

A crea una transazione con un multi signature e threshold tra A, B e J con threshold a 2.

Cosa potrebbe andare storto? A potrebbe non ricevere il bene o B potrebbe mentire, A potrebbe volere un rimborso. In questi casi J decide in favore di A o B.

Micro payments : voglio pagare una piccolissima parte ad un vendor ma questo tipo di pagamenti potrebbero costare più a livello di mance rispetto al pagamento stesso. (esempio del pagamento di una chiamata di un ora)

In questo caso sfruttiamo il lock_time con una multisignature che ha bisogno di entrambe le firme. Succede che A deve fare un micropagamento a B ad ogni minuto che passa e, quindi, fa una sorta di transazione dove A manda 1 a B e B ritorna ad A 59. Questa transazione è firmata con la sola chiave di A e, quindi, non è ancora accettabile dalla blockchain. Stessa cosa accadrà a 2 minuti e così via...A ogni minuto paga quel minuto con questa specie di transazione e B li stora. B quando riceve l'ultima transazione li firma e li manda alla blockchain. Ricordiamo che ognuna di queste sottotransazioni sono tentativi di double spending, quindi B firmerà solo l'ultima in pratica. B è tranquillo perché tanto in qualunque momento può firmare l'ultima transazione e riscattare il suo denaro. Ma cosa accade se B non collabora e non vuole firmare l'ultima transazione di fatto trovando uno stallo perché B non riceve i soldi ma A non riceve nemmeno il resto indietro.

Per risolvere questo problema usiamo il lock_time, quando A locca i soldi tra A e B, B deve mandare una transazione in favore di A che gli ritorna i soldi nel caso in cui passa il lock_time. Per questo motivo B è invogliato a firmare perché altrimenti diventa valida quella di rescue e quindi B perde tutti i soldi. Una volta che B firma l'ultima

transazione, quella di rescue non è più valida perché diventa un tentativo di Double-Spending.

Freezing Funds: una sorta di assegna post datato.

Pay to script hash: è uno script nato dopo dalla community.

Tutte queste funzionalità non fanno altro che complicare naturalmente il classico script di pagamento. Immaginiamo che questi script siano gestiti dal vendor e per fare questo pagamento si deve mandare l'hash dello script. Il venditore deve creare prima lo script e poi pubblicare l'hash perché se così non fosse, il vendor potrebbe cambiare lo script e quindi "chettare". Lo scriptPubKey rimane piccolo perché contiene l'hash dello script mentre lo script vero e proprio è contenuto nello scriptSig.

Il vendor per spendere i soldi dovrà fornire lo script completo e poi tutto il necessario.

Parliamo di blocchi.

Bits rappresenta la difficoltà dell'hash puzzle da risolvere dei blocchi prima di questo.

Nonce è un numero casuale messo dal miner per risolvere l'hash puzzle.

Merkle Root. Le transazioni sono le foglie dell'albero. I merkle tree supportano le operazioni di proof of membership. Per fare questo, devo fornire gli hash dei suoi sibling dalla foglia alla root.

Nel blocco non abbiamo dunque la copia di tutte le transazioni, vedremo che, in realtà, solo i full node contengono tutto il merkle tree. I light node tengono solo il block header. I light node quindi chiederanno ai full node di ricevere i dati necessari alla proof of membership.

Durante la risoluzione dell'hash puzzle si modifica solo la nonce (o al più tutto il discorso che dicevamo del merkle tree che ha a altre foglie per nonce)

Special transaction

Coinbase è un field libero in cui si può mettere quello che si vuole

Come si crea una nuova transazione

Gossip protocol

A manda la sua transazione e i blocchi li storano nella transaction pool. Fanno vari controlli:

1. La transazione deve essere valida (no double spending). Per velocizzare, i nodi possono tenere una sorta di cache con gli UTXO non spesi. Questa cache non può essere scaricata da nessuna parte, si deve verificare da soli.
2. Script deve essere tra quelli whitlistati (deve essere valida)
3. Non deve essere già nella pool.
4. Non deve essere in conflitto con un'altra transazione nella pool

09/04/2026

Annuncio di un nuovo blocco

La propagazione si rallenta perché molti nodi usano VPN o la Onion network.

I full node hanno una completa copia della blockchain e possono propagare transazioni e nuovi blocchi. Inoltre, per ottimizzare il tutto, possono tenere un UTXO cache nella ram così da velocizzare il controllo di double spending.

I lightweight node possono contenere solo gli header dei blocchi e possono chiedere le proof dei merkle root.

I limiti di bitcoin

I sistemi di firma ECDSA dovranno essere sostituiti con crittografia post-quantum.

Hard fork sono blocchi con nuove features che, però, non sono considerati validi da tutti gli altri. Tradotto, creeranno una nuova catena che sarà continuata da tutti i miner aggiornati mentre un'altra catena sarà continuata dagli altri miner con il software non aggiornato. Questo è chiaramente molto difficile. Esempio: aggiungo un nuovo sistema di firma.

Soft fork creano una situazione in cui si possono aggiungere blocchi non molto validi. Un esempio fu il caso di pay to script hash. In pratica le nuove regole creano blocchi che sono comunque validi per i vecchi miner ma è possibile che vecchi nodi creino blocchi non validi per i nuovi miner.

Sistema per storing bitcoin

I wallet sono comodi per gestire multipli address dato che sarebbe bene avere indirizzi diversi per transazioni diverse.

Base58 coding

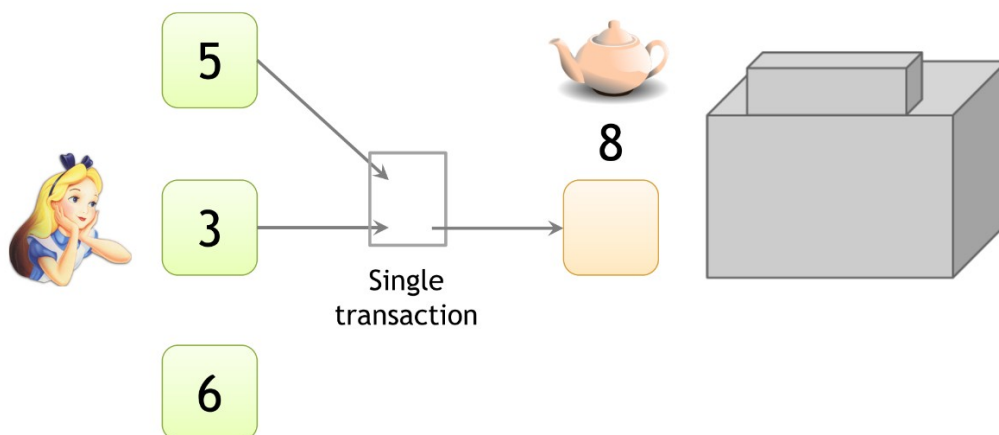
Ci sono dei trick per ottenere indirizzi personalizzati. Sì, anziché fare ogni volta nuove x e, quindi, nuove esponenziazioni, moltiplico per G.

Utile avere un hot storage (per le spese quotidiane) e un cold storage (per i risparmi ecc.) però è comodo perché è facile spostare da hot a cold ma non viceversa e questo ci piace dato che sono i "risparmi". Il cold storage rimane offline in questo caso. Per nascondere la mia identità faccio prima un trasferimento verso una nuova identità da inserire poi nel cold storage però a questo modo dovrei aprirlo. Questo può essere gestito in altri modi.

14/04/2026

Bitcoin non è anonimo anche se uso un nuovo address per ogni transazione.

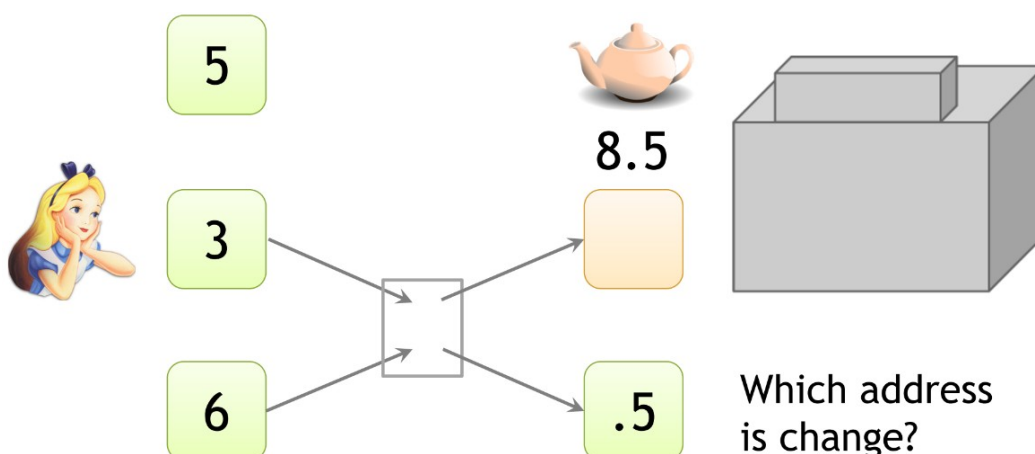
Immaginiamo che A voglia comprare una tegliera e A ha tanti UTXO dove contiene i bitcoin e ognuno di questi UTXO è firmato con una differente chiave.



Vedendo 5 e 3 vediamo che queglii UTXO appartengono alla stessa entità anche se non sappiamo chi questa entità è.

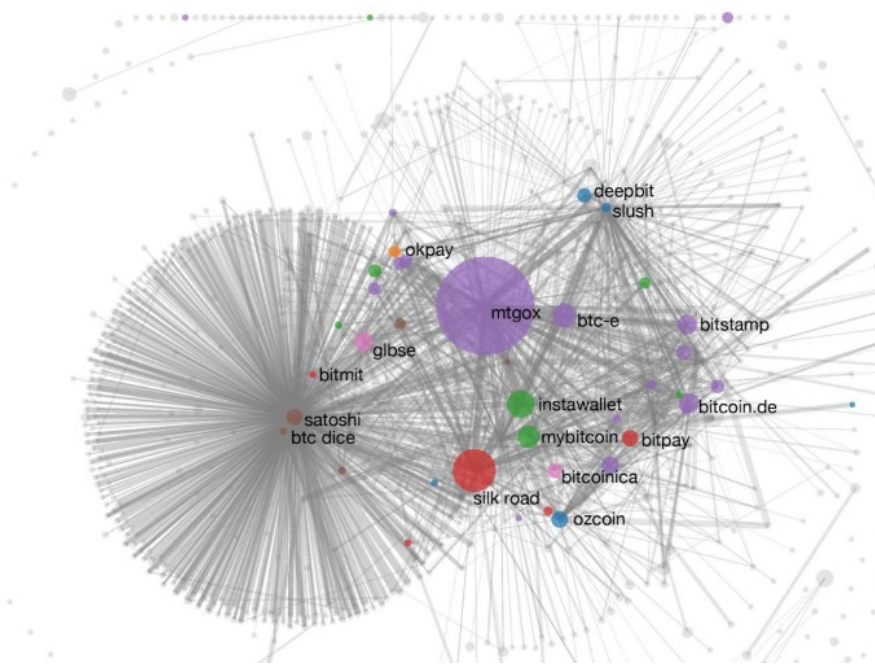
Continuando a linkare possiamo cominciare a capire chi è chi e da dove viene cosa. Otteniamo un clustering di indirizzi che appartengono alle stesse entità.

Anche il pagamento della cifra che non è intero ci dà problemi:



Riusciamo a collegare non solo 3 e 6 ma anche .5. Ma come sappiamo che è proprio il .5 ad appartenere ad Alice, potremmo complicare le cose facendo sì che il resto sia maggiore del pagamento stesso? Complicando molto le cose, possiamo un minimo garantire anonimato ma non è detto che questo venga realmente fatto. Infatti, il pagamento dipende da come funziona il Wallet, quindi non è lui a decidere come realmente fare le cose. Anche se implementassimo il software in un certo modo, questo sarebbe pubblico e, quindi, più o meno prevedibile. Aggiungendo questo, arriviamo a trovare un grafo maggiore ancora di conoscenza.

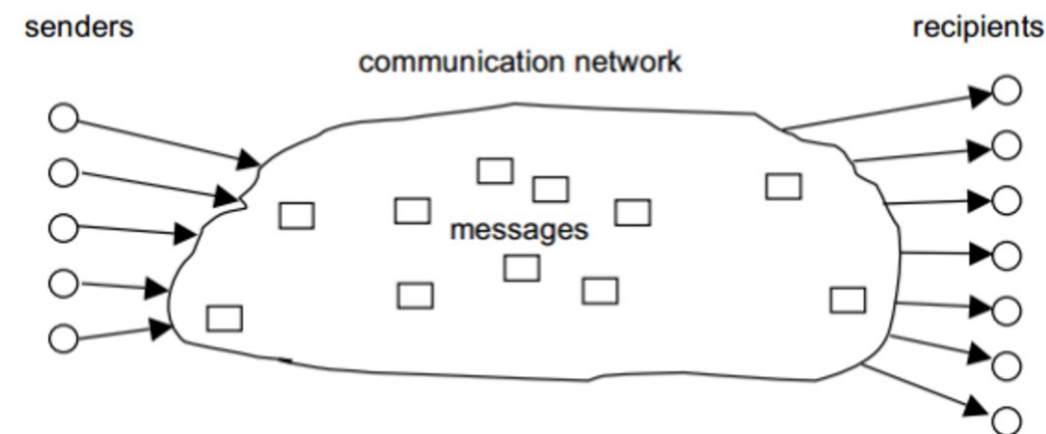
Il problema è adesso capire chi è quella specifica entità. Questo può essere fatto tramite guess anche in base alla quantità di denaro spostato.



Ma questo come attacca il singolo individuo? Pensiamo al social engineering, le persone non hanno realmente idea di quello che fanno. Qualunque cambio da bitcoin a monete reali/beni, allora deanonimizziamo bitcoin.

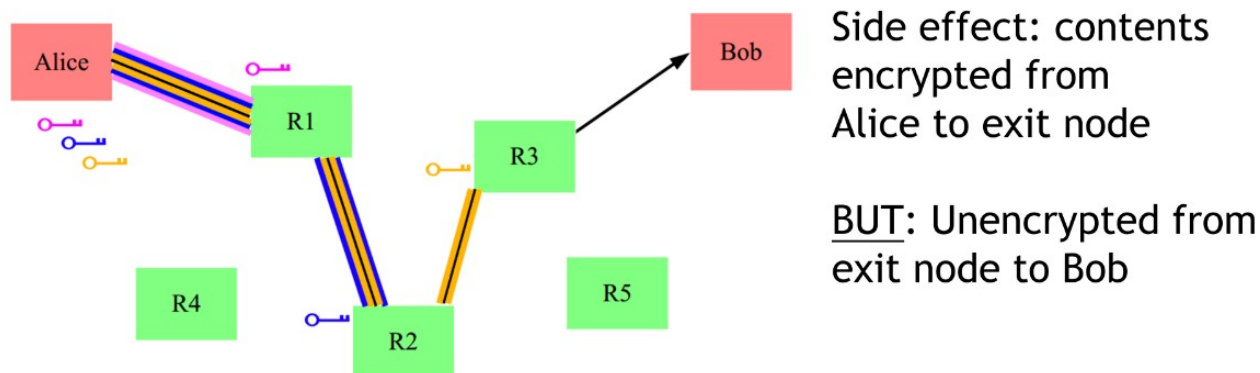
Anche la comunicazione che avviene tramite la rete crea problemi all'anonimato. Più complicati certo, ma può valerne la pena.

Usiamo Tor:



Vorremmo una cosa del genere cioè un sistema che non sa da chi arriva il messaggio. Abbiamo chiaramente bisogno che questo sistema sia usato da tante persone perché altrimenti sarebbe ovvio chi manda e chi riceve.

Sender A e Receiver B. A quando manda un messaggio, sceglie un certo numero di macchine e decide un certo percorso. Cifra il messaggio in maniera tale che questo sia cifrato a cipolla così che ogni pezzo della comunicazione possa eseguire una certa operazione.



Quello che potrebbe accadere è che uno di questi nodi potrebbe essere malevolo. Ma se almeno uno è onesto, l'anonimato è preservato.

Possono esserci vari casi:

1. Se R2 e R3 sono malevoli. R1 sa da chi viene il messaggio ma questa info non la inoltra. R2 e R3 possono conoscere B ma non A.
2. Se R1 e R3 sono malevoli. R2 semplicemente ignora l'identità di A inoltrata da A.
3. Se R1 e R2 sono malevoli. R3 non inoltra il nome di A.

Più server ci sono, più è probabile che il sistema sia sicuro però più server ci sono, più la latenza aumenta.

Usiamo mixing network

Un approccio utile è quello di avere un intermediario per noi che manda chunk di messaggi. Questo è esattamente quello che fa la banca. Collecta messaggi e poi li manda a chi deve mandarli.

Quello che fanno i wallet online non è propriamente questo. Per fare questa cosa, ci si deve creare il proprio mixing server. Per funzionare dovrebbe promettere di non tenere traccia dei pagamenti e non dovrebbe chiederti l'identità. Questa cosa non sembra per niente legit...chi mi assicura che non scappano con i miei soldi?

In genere, un sistema di wallet deve gestire record e richiede un'identità, quindi non abbiamo anonimato.

Supponiamo di voler risolvere questo problema del sistema di Mixing in maniera tale che possiamo usarlo:

1. Non dovremmo usare un singolo mixing server ma ce ne servirebbe più di uno. Così dovresti romperli tutti per sapere cosa sta accadendo.
2. Le transazioni devono essere uniformi altrimenti sarebbero sgamabili. Vanno bene anche dei possibili chunk uniformi.
3. Il client side deve essere automatizzato e uniforme. Così che tutti facciano la stessa cosa e nessuno possa essere deanonimizzato.
4. Come funziona il mixer? Chi lo paga? O guadagnano su una percentuale della transazione ma questo creerebbe tanti micropagamenti e, quindi, sarebbe deanonimizzato. L'altro approccio è una all-or-nothing. Su 10 chunk uno viene sempre dato al mixer.

Nessuno lo fa. Gg wp go next ff 15.

16/04/2026

Differenza tra hot storage e Cold storage.

Vorremmo che il cold storage sia offline e più sicuro. Uno spostamento da cold a hot è necessario aprire il cold storage e questo è okay. Per depositare abbiamo solo bisogno della public key del cold storage. Tuttavia, questo però si contrasta con il principio di avere sempre nuove coppie di chiavi per ogni transazione a mio favore.

Per risolvere il problema, utilizziamo con uno specifico algoritmo due seed:

1. Address gen info che va nell'hot storage
2. Private key gen info che va nel cold

Dando l'indice i , io genero con il primo l'indirizzo e con il secondo la chiave privata. Per fare questo con ECDSA avremo che:

1. Address gen info contiene k e G^y
2. Private key gen info contiene k e y
3. Per calcolare la i -esima chiave privata usiamo $x_i = y + h(k||i)$
4. Per quella pubblica $G^{(x_i)} = G^{h(k||i)} * G^y$.

Anche conoscendo l'identità della i -esima chiave, non posso linkarlo all' i -esima + 1 grazie alla pseudo randomicità della funzione hash. Diciamo unlinkability questa proprietà.

Il cold storage è in genere un tamper proof device che è un dispositivo che anche se venisse rubato, richiederebbe comunque una password per accedere ecc. Come le smart card.

Possiamo anche memorizzare una password a memoria così da poter generare a partire da questo quello che vogliamo. Però, poiché il nostro cervello è pigro, spesso le password sono molto semplici nel senso che sono riconducibili a qualcosa di reale e, quindi, vulnerabili a guessing attack.

L'idea è che a partire dalla password possiamo generare il segreto come il suo hash. Usando hash parecchio lente, posso rallentare gli attacchi.

Tutti quelli che usano brain wallet, potrebbero essere vulnerabili a password cracking.

Potrebbe avere senso gestire più posti dove memorizzare la secret key, però non è conveniente questa cosa, dobbiamo proteggere più cose però non dovendo avere un singolo point of failure. Il secret sharing ci porta a poter gestire più chiavi e tollerando un certo numero di fallimenti.

Il pay to hash che gestisce il secret sharing potrebbe avere svelare cose del tipo chi era d'accordo a fare un certo pagamento ecc.

Divido il segreto in N pezzi tramite lo xor o somma (vedremo dopo)

Threshold key, abbiamo bisogno di $t+1$ chiavi per confermare il pagamento.

Sistema di Shamir, usiamo delle rette per threshold a 1, parabole per threshold a 2 ecc. E, in generale, equazioni di grado 3 per threshold a 3 e così via.

L'idea non è quello di ricostruire il segreto, bensì quello di creare firme parziali che combinate danno la firma parziale.

21/04/2026

L'idea è che usare un mixer decentralizzato tramite lo schema di firma che abbiamo visto con MDR, possa garantire l'anonimato

Sfrutteremo ciò che ha proposto Greg Maxwell che si chiama Coinjoin.

Alla fine, vedremo solo che qualcuno ha voluto pagare qualcun altro ma non riusciamo a capire chi ha pagato chi. Più persone usano questo sistema, più l'anonimato è garantito(?)

Se vogliamo fare un pagamento, troviamo un certo numero di altre persone che vogliono fare il pagamento. Con questi scambiamo address di destinazione e di partenza. Si costruisce la transazione e poi tutti la firmano e la fanno girare nella rete bitcoin

Problemi:

Dobbiamo trovare le altre persone che, però sapranno chi siamo.

Se le persone però non firmano dopo aver accettato, abbiamo un DDOS.

La soluzione potrebbe essere collegarsi a Tor, scambiarsi gli input, uscire e rientrare da Tor e poi scambiare gli output.

Ci sono state altre soluzioni come il PoW per fare coinjoin e quindi renderlo costoso l'attacco.

Non ha più senso creare delle valute che sono di base anonime?

Nuovo PPT

In contesto crittografico vogliamo dimostrare di avere determinate conoscenze. Come faccio a dimostrare di avere queste conoscenze senza effettivamente rivelare l'informazione?

Che prova vogliamo dare? Cosa intendiamo per dimostrazione e cosa vuol dire che la prova non deve fornire informazioni?

Una dimostrazione è qualcosa che ci convince della validità di un asserto. In matematica questo è statico, nel nostro caso la dimostrazione è diversa. Nel nostro caso sarà un processo dinamico in cui avremo un prover P e un Verifier V che interagiscono tra loro. La cosa è in realtà probabistica e potrebbe cambiare di volta in volta la prova.

Il prover vuole dimostrare che la prova è vera e il verifier dovrà verificare che lo sia.

Verificare è più facile che provare. Questa asimmetria è dimostrata perfettamente dai problemi NP che abbiamo già visto a crittografia. Il Verifier deve essere limitato e il Prover può anche essere unbounded o sembrarlo nel caso in cui conosce più cose.

Una prova per essere una buona prova deve essere:

1. Completa: P dovrebbe riuscire a convincere V della validità degli asserti veri.

2. Soundness: P disonesto non dovrebbe essere in grado di convincere V della validità di un asserto falso.

A questo punto, cosa è la conoscenza? Cosa vuol dire avere un guadagno in termini di conoscenza?

Supponiamo di avere A che parla e B che ascolta. A non guadagna conoscenza ma B in teoria acquisisce qualcosa. Tuttavia, non è detto che B stia realmente acquisendo conoscenza che magari non aveva già prima.

Diciamo che B acquisisce conoscenza se la sua capacità computazionale in qualche modo viene arricchita. Acquisisce conoscenza solo se riceve il risultato di una computazione che non era in grado di fare da solo. P,V è un proof system interattivo per il linguaggio L se V è polinomialmente limitato e le seguenti condizioni sono verificate:

Completeness:

Completeness: P dovrebbe essere in grado di convincere V della validità di asserti veri.

Soundness: Un P disonesto non dovrebbe essere in grado di convincere V della validità di un asserto falso.

Informalmente, un sistema di prove interattive (P,V) per un linguaggio L è detto zero knowledge, se qualsiasi cosa è calcolabile interagendo con P su input $x \in L$, è calcolabile a partire da x soltanto.

Supponiamo che noi siamo il verificatore e che il prof sia il prover che ha un modo per distinguere inequivocabilmente da quale bottiglia proviene un certo bicchiere d'acqua. Noi riempiamo un bicchiere, se il prof non indovina, allora viene rigettato, se indovina (ha beccato una probabilità $\frac{1}{2}$) lo facciamo riprovare: se sbaglia allora gg wp ez rigettato, altrimenti potrebbe aver indovinato nuovamente $\frac{1}{2} * \frac{1}{2}$, quindi $\frac{1}{4}$. Se iteriamo il processo, la probabilità che lui bechi sempre più volte la risposta corretta è sempre più bassa.

Il problema è che come sappiamo che non eravamo collusi noi e il prof?

Il protocollo è un proof system corretto perché garantisce le due proprietà.

Definiamo il concetto di view che non è altro che la variabile aleatoria che contiene tutti i messaggi che sono stati scambiati tra P e V durante il loro scambio. Possiamo anche supporre che P e V abbiano una cosa privata h ma che non sono in view perché sono delle sorte di preconnoscenze.

Diciamo che (P,V) è un interactive proof system che fornisce perfect zero knowledge per il linguaggio L se:

$\exists M$ PPTM, tale che

$\forall x \in L, a > 0, \forall h$ tale che $|h| < |x|^a$

$M(x, h)$ e *view* sono distribuite in modo identico.

Nell'esempio fornito la macchina M non fa altro che ricostruire la *view* senza dare la conoscenza ma sapendo solo l'input x e il segreto del verificatore h .

Possiamo usare questo sistema per altri problemi (no bottle of water)